# Low-Cost ESP32 Smart Insole for Gait Monitoring and Parkinson's Gait Screening with a Lightweight 1-D CNN

**Abhigya Mahawar, Aneesh Bansal, Shrihan Yekkele**

*Embedded Systems Course Project, October 2026*

***Abstract.*** *Gait carries early signs of neurological disease, but clinical gait laboratories are costly and rarely available for routine or remote monitoring. This paper presents a low-cost smart insole built around an ESP32 microcontroller and six force-sensitive resistors (three per foot, at the toe, midfoot and heel), sampled at 100 Hz and streamed over Bluetooth to a PC recorder. Recordings are uploaded to a FastAPI and SQLite backend that computes gait metrics (step count, cadence, variability and left-right asymmetry), draws foot-pressure maps and shows a screening score on a web dashboard. For gait recognition, five deep models (CNN, LSTM, CNN-LSTM, Transformer and a simplified Temporal Fusion Transformer) were trained on the public PhysioNet gaitpdb data (306 recordings, 165 subjects) to separate Parkinson's disease from control walking, using subject-wise five-fold cross-validation and a leave-one-study-out test. A 37,090-parameter 1-D CNN performed best or jointly best (macro F1 0.759 and AUC 0.866 with three sensors per foot) and was deployed in the dashboard, where scoring a 60 s recording takes about 36 ms on a two-core cloud instance. Performance fell on an unseen study (macro F1 0.706), and the model has not been validated on our own insole: scores on our marching-in-place recordings proved unstable, so the dashboard shows the output only as an indicator and not as a diagnosis. The model runs in the cloud and not on the microcontroller. Quantisation, on-device latency and power were not measured and are listed as future work.*

***Keywords:*** *smart insole; gait analysis; force-sensitive resistor; ESP32; Parkinson's disease; 1-D convolutional neural network; PhysioNet gaitpdb; remote patient monitoring; embedded systems.*

## 1. Introduction

### *1.1 Background and motivation*

Walking is controlled by the brain, the spinal cord, the nerves and the muscles together, so the way a person walks changes early in many neurological conditions. In Parkinson's disease, stride time becomes less regular and steps shorten, and in stroke the two legs load unequally. A gait laboratory measures these changes with force plates and motion capture, but the equipment is expensive and confined to a few centres, and a patient is observed for a few minutes in an unnatural setting. A wearable insole that records the pressure under each foot can capture the same loading pattern at a fraction of the cost, in a clinic corridor or at home.

The reference work for this project, Yan et al. [1], built a low-cost wearable gait detector from two force-sensitive resistors (FSRs) and an inertial sensor per insole, an Arduino Uno and a mini-PC, and detected five gait events with 99.66% accuracy. Our project follows that idea in three directions: a foot-pressure-only insole on an ESP32, a comparison of modern deep learning models on public gait data, and a cloud dashboard that turns recordings into metrics a clinician can read.

### *1.2 Real-world importance*

Parkinson's disease is diagnosed from clinical examination, and the severity is followed by repeated clinic visits, so a cheap and objective measurement that can be repeated at home would help to follow a

patient between visits. Falls and reduced mobility are leading causes of loss of independence in older adults, and loading asymmetry or rising step-to-step variability are signs that a physiotherapist can act on. Remote monitoring also matters where specialists are far from the patient. A low-cost insole with a cloud dashboard is a step towards these uses, provided its limits are stated clearly. Section 9 gives those limits for this prototype.

### *1.3 Why an Arduino-compatible microcontroller with machine learning*

The insole needs a device that reads several analogue sensors at a fixed rate, runs for hours from a small battery or power bank, sends data wirelessly and costs little. The ESP32 DevKit V1 used here is programmed from the Arduino IDE with the Espressif core, so it is an Arduino-compatible board in the sense of this course. It has a 12-bit ADC with many input channels, two cores (one can sample at a fixed rate while the other handles communication), built-in Bluetooth and WiFi, and a cost of a few hundred rupees. Machine learning is a good fit for two reasons. First, hand-written rules for gait, such as the rule-based calibration in [1], need tuning for each sensor placement and gait type, while a learned model can take the raw loading signal. Second, a small network is cheap to run: the CNN selected here has 37,090 parameters and needs about 1.26 million multiply-accumulate operations per two-second window (Section 6.6), which is within the range that TinyML toolchains target. In this prototype the network runs on the cloud backend and not on the microcontroller, and Section 6.6 says what moving it on-device would involve.

### *1.4 Scope of the project*

The project covers the following.

- A six-FSR insole prototype on breadboards, with an ESP32 that samples at 100 Hz and streams over Bluetooth.
- A PC recorder and a cloud application (FastAPI, SQLite and a web dashboard) for gait metrics, foot-pressure maps, alerts and a screening indicator.
- A comparison of five deep models for gait recognition on PhysioNet gaitpdb, with a robustness test across recording studies.
- A cloud-connected dashboard for remote monitoring, demonstrated with team volunteers' recordings.

The following are outside the scope: inertial sensors (the prototype is pressure-only), inference on the microcontroller, stroke evaluation (listed as future work), clinical trials and any diagnostic claim.

## 2. Literature Survey

The survey is organised around four questions that this project raises: how pressure insoles are built and evaluated, how deep models detect Parkinson's gait, how such sensing is joined to a platform a clinician can use, and whether the models can run on a microcontroller. Eight recent works form the core of the survey (Table 1). Two review articles [9], [10] and the dataset sources [11]-[15] give background.

### *2.1 Wearable gait sensing and insole design*

The base paper, Yan et al. [1], built a low-cost wearable gait detector from two FSRs (heel and toe) and a 9-axis inertial sensor in each insole, with an Arduino Uno sampling at 300 Hz and an x86 mini-PC running a multilayer perceptron. A rule-based calibration filter that uses the FSR loads and the previous

gait phase corrected the network's mistakes and raised the detection of five gait events from 96.25% to 99.66%, with a 32 ms mean time error against a motion-capture reference on five subjects walking on a treadmill. The design shows that very cheap sensors can follow gait phases, but it targets healthy walking, needs the inertial sensors and a PC-class controller, and does not classify disease.

Movrin et al. [7] describe a closer match to our hardware: an insole with five FSRs, an nRF52840 microcontroller sampling at 100 Hz, Bluetooth Low Energy and a phone application. In a test with one subject, the left and right insoles gave matching trends, and the hardness of the insole material changed the peak sensor readings. The result is a reminder that the mounting of an FSR affects what it reports, which we also saw in the unit-to-unit differences between our pads. The work stops at data collection and has no learning stage.

The review by Latsch et al. [9] compares insole sensing technologies (resistive, capacitive, piezoelectric, triboelectric, optical and others) and the way studies place sensors and validate them. It finds that many studies lack proper sensor calibration, verification against gait references and validation with human participants, and that soft-tissue stiffness is a large source of uncertainty. This matters for a resistive FSR insole such as ours: its absolute force is not calibrated, so our analysis uses per-recording normalised signals and does not report forces in newtons.

### *2.2 Deep learning for Parkinson's gait*

Most deep learning work on Parkinson's gait uses the public gaitpdb recordings of vertical ground-reaction force [11]-[15]. El Maachi et al. [2] feed each of the foot-sensor signals into its own one-dimensional convolutional branch, 18 in all, and merge them in a fully connected classifier, reporting 98.7% accuracy for detecting Parkinson's disease and 85.3% for predicting severity on the UPDRS scale. Nguyen et al. [3] replace the convolutions with a temporal Transformer followed by a spatial Transformer, which keeps the model smaller, and report 95.2% accuracy on the same data. Meral and Ozbilgin [4] tune CNN and LSTM models with Bayesian optimisation on windows of 5 to 25 s and report up to 99.42% for detection (LSTM, 10 s windows) and 98.24% for Hoehn-Yahr staging.

These accuracies are far above the 0.79 that we obtain with a subject-wise split and 0.74 on an unseen study (Section 9). The numbers are not directly comparable: window lengths differ, the sensor sets are larger, and the protocols differ. How recordings of the same person are divided between training and test data decides whether a score reflects new people, and the summaries of [2]-[4] do not make that protocol clear enough for us to compare against them. We therefore use a strict subject-wise protocol and a leave-one-study-out test, and we do not claim to match these figures.

Park et al. [5] work on a real insole instead of the force-plate dataset. Their CNN processes each foot separately with alternating spatial and temporal convolutions and was trained on slow-gait data from 29 participants (13 healthy, 9 elderly and 7 with Parkinson's disease). It reached about 90% accuracy with 8 of the 32 sensors of the insole, roughly 10 percentage points above a conventional CNN. The finding that fewer plantar sensors can be enough supports a three-sensor design, although the cohort is small and the insole is commercial-grade.

### *2.3 Platforms that connect the insole to a clinician*

Boucharas et al. [6] present a smart insole platform aimed at Parkinson's patients. The insole pair carries 16 pressure sensors, an accelerometer and a gyroscope; a mobile app prepares the data for a cloud platform with a web application, and open communication protocols let components be replaced without

changing the architecture. The platform supports AI algorithms and visualisations such as heat maps, and is proposed as a digital biomarker for diagnosing and following the disease. The paper describes the framework and does not report validation metrics. Our dashboard follows the same idea on a smaller scale: insole, recorder, cloud service and web view, with a foot-pressure map and trend views.

### *2.4 Embedded machine learning on microcontrollers*

Madhiha et al. [8] built a stand-alone gait diagnosis unit on an ESP32 with two MPU6050 motion sensors, running an 8-bit quantised neural network that classifies five gait types, together with a K-means anomaly score. Trained on about 15 minutes of data from five volunteers, it reached about 92% accuracy, produced a result in roughly 95 to 96 ms from 3 s of data, and cost about US$7.30. The survey of Abushahla et al. [10] reviews the quantisation methods, numerical formats and microcontroller platforms (ARM, RISC-V and chips with neural accelerators) used to fit networks onto such devices; 8-bit quantisation, as used in [8], is a common first step. Both sources concern motion sensors, so on-device inference from foot pressure remains to be shown. In this work we only estimate what it would need (Section 6.6).

**Table 1.** Core works of the survey.

| Ref. | Work | Method | Reported result | Relevance and limit |
|---|---|---|---|---|
| [1] | Yan et al., 2021 | 2 FSRs and a 9-axis IMU per insole, Arduino Uno plus mini-PC, MLP with a rule-based calibration filter | 99.66% gait-event accuracy, 32 ms mean time error, 5 subjects on a treadmill | Base paper. Healthy walking, needs a mini-PC and IMUs, no disease classification |
| [2] | El Maachi et al., 2020 | 18 parallel 1-D CNN branches, one per foot sensor, on gaitpdb force signals | 98.7% Parkinson's detection, 85.3% severity (UPDRS) | Strong on gaitpdb, all force channels, no hardware |
| [3] | Nguyen et al., 2022 | Temporal then spatial Transformer for 1-D gait signals on gaitpdb | 95.2% Parkinson's detection | Transformer on gait signals, protocol not clear from the summary, no hardware |
| [4] | Meral and Ozbilgin, 2025 | CNN and LSTM with Bayesian hyperparameter search on gaitpdb, 5 to 25 s windows | LSTM 99.42% detection (10 s window), 98.24% Hoehn-Yahr staging | Very high scores, long windows, no deployment |
| [5] | Park et al., 2026 | CNN with per-foot spatial and temporal convolutions on a 32-sensor smart insole | About 90% accuracy with 8 of 32 sensors, 29 participants (7 with Parkinson's) | Fewer plantar sensors can work, small cohort, commercial-grade insole |
| [6] | Boucharas et al., 2022 | Insole with 16 pressure sensors and IMU, mobile app, cloud platform and web app | Framework description, no validation metrics reported | Closest to our platform, larger sensor set, no results |
| [7] | Movrin et al., 2025 | 5-FSR insole, nRF52840, BLE, 100 Hz, phone app | Left and right trends matched, TPU hardness changed peak readings (1 subject) | Closest hardware, no machine learning |
| [8] | Madhiha et al., 2024 | ESP32 with two MPU6050 sensors, int8 neural network and anomaly score | About 92% accuracy, 95 to 96 ms per 3 s window, about US$7.30 | On-device ESP32 inference, motion sensors, five healthy gait types |

### *2.5 Key research gaps*

- **Hardware and learning are rarely joined for pressure insoles.** The insole prototype [7] stops at data collection, and the deep models for Parkinson's gait [2]-[4] use recorded datasets and are not connected to a low-cost build. The microcontroller classifier [8] uses motion sensors, not foot pressure.

- **Evaluation protocols differ and are rarely strict.** Reported gaitpdb accuracies of 95 to 99% [2]-[4] are not comparable with a subject-wise, cross-study evaluation, and how a model holds up on a recording setup it has not seen is seldom reported.
- **Low-cost insoles are seldom calibrated or validated.** The review [9] finds that many insole studies lack calibration, reference verification and human validation, and the small clinical cohorts such as [5] limit what can be concluded.
- **Platforms are described more often than they are evaluated.** The framework in [6] has the right parts (insole, app, cloud, web) but no validation results, and a usable system also needs per-session metrics, alerts and a clear statement of what the score means.
- **Small sensor sets need testing.** The study [5] suggests that fewer sensors can keep accuracy. We test this directly with 2, 3 and 8 sensors per foot (Section 9.1).
- **Edge deployment claims need measurements.** Work on microcontroller inference [8], [10] differs in whether latency, memory and power are reported. In this work we state which of these we measured and which we did not.

This project responds to these gaps in three ways: a complete low-cost path from six FSRs to a dashboard, a strict subject-wise and cross-study comparison of five model families on three sensor sets, and a plain statement of what was measured on the hardware, what was only calculated and what was not done.

## 3. Problem Statement and Objectives

Clinical gait analysis is expensive and not available for frequent or remote use. The problem is to build a low-cost insole that records foot pressure, to test whether deep learning can recognise a pathological gait pattern from such signals, and to present the result in a form a clinician can use, while being honest about how far the evidence goes. The objectives are as follows.

- Build a six-FSR insole with an ESP32 that samples all channels at 100 Hz without dropped samples and streams them wirelessly.
- Record the six channels to files and upload them to a cloud application that stores sessions, computes gait metrics and raises alerts.
- Compare a CNN, an LSTM, a CNN-LSTM, a Transformer and a simplified Temporal Fusion Transformer on Parkinson's versus control classification, using subject-wise splits so that no person is in both training and test data.
- Test how each model holds up on a recording study it has not seen, and how many sensors per foot are needed.
- Deploy the best small model in the dashboard as a screening indicator with a clear disclaimer, and measure the time it takes.
- Report limits: what was measured on the hardware, what was measured only on public data, and what was not measured.

## 4. System Architecture

Figure 1 shows the system in three parts. In part A, six FSRs in voltage dividers feed the ADC of an ESP32, whose sampler task sends 100 Hz rows over Bluetooth to a PC recorder that writes one CSV file per walk. In part B, the file is uploaded to a FastAPI backend, which computes gait metrics and, in parallel, runs the machine learning pipeline; results are stored in SQLite and shown on a web dashboard. In part C, the CNN used by the pipeline was trained offline on PhysioNet data, and its 37,090 weights are copied into the backend as a 150 KB file.

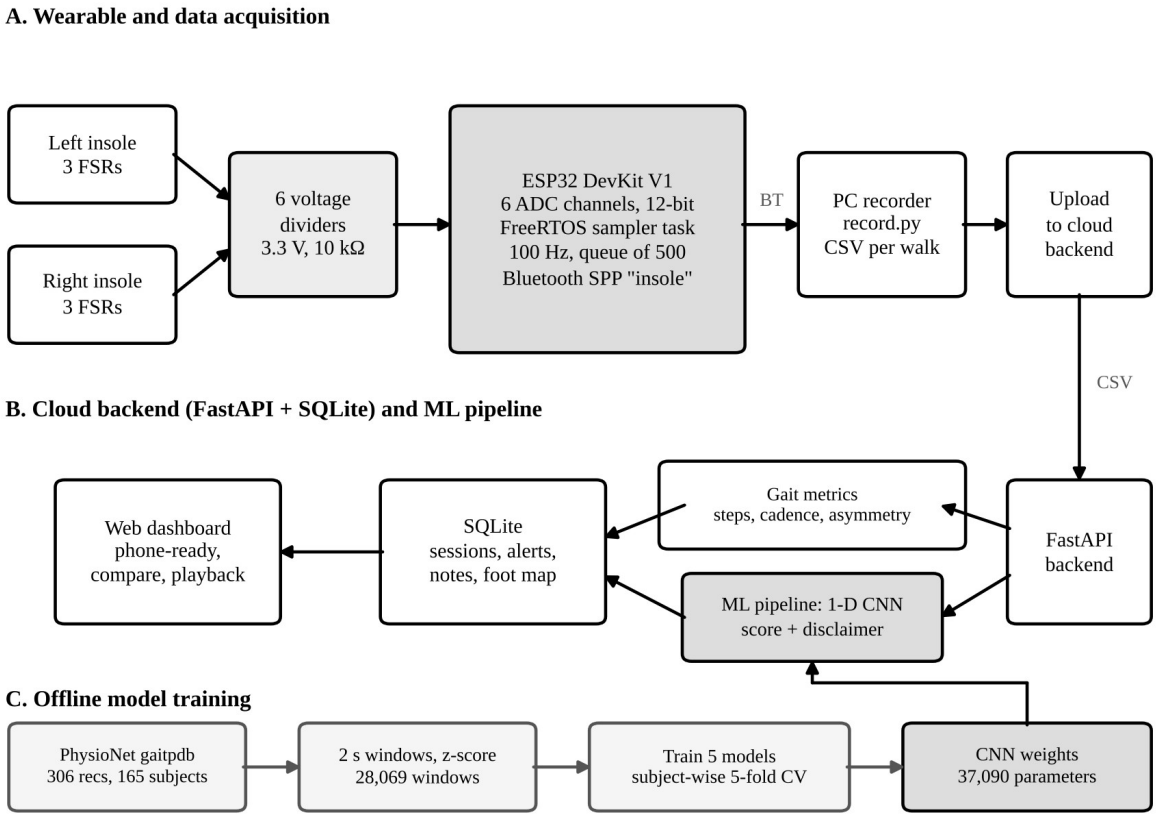



**Figure 1.** System block diagram: wearable and acquisition (A), cloud backend and ML pipeline (B), offline training (C).

Figure 2 gives the circuit. Each channel is a voltage divider with a 10 kΩ resistor to the 3.3 V rail and the FSR to ground, and the junction goes to an ADC pin. The ADC reads about 4095 counts when the pad is unloaded, and the value falls as the pad is pressed. The same circuit is repeated for the six pins listed in the table of Figure 2.

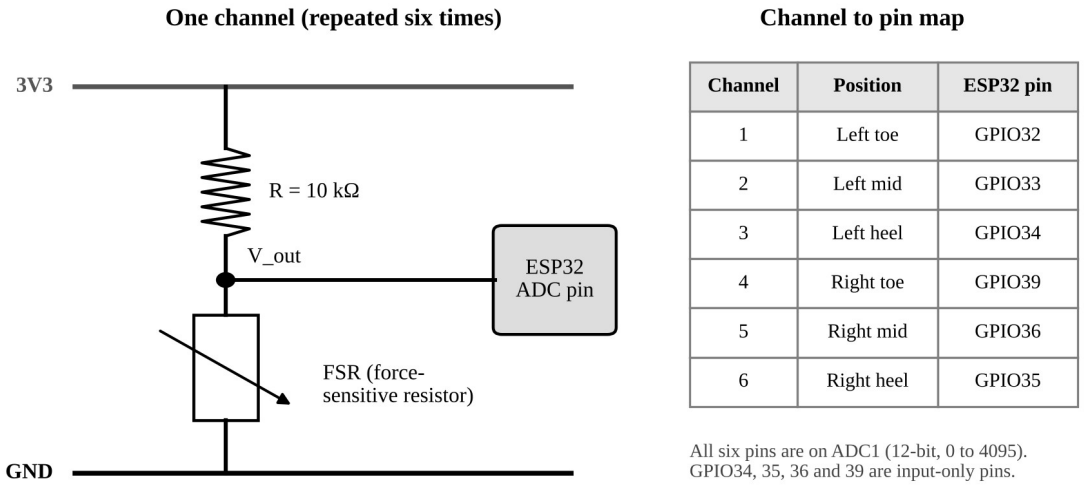



**Figure 2.** Circuit diagram of one voltage-divider channel and the channel to pin map.

## 5. Hardware Design and Connections

### *5.1 Component details*

**Table 2.** Components of the prototype.

| Component | Qty | Role |
|---|---|---|
| ESP32 DevKit V1 (38-pin) | 1 | Microcontroller. 12-bit ADC, dual core, Bluetooth Classic. Programmed in Arduino IDE 2.3.6 with the Espressif ESP32 core |
| Force-sensitive resistor (FSR) | 6 | Pressure sensing, three per foot at toe, midfoot and heel. Resistance falls as load rises |
| Resistor 10 kΩ | 6 | Fixed leg of each voltage divider |
| Breadboard, full size | 2 | A for the left foot, B for the right foot. No soldering |
| Jumper wires | - | Male-to-female wires link the ESP32 to the boards, ordinary jumpers link the dividers |
| USB cable and power bank | 1 | Power and programming. Data leave over Bluetooth |

The estimated budget for the whole prototype was INR 1,500 to 2,490, set at the start of the project.

### *5.2 Hardware connection diagram*

Figure 3 shows the connections. The ESP32 stays off the breadboards, because the 38-pin board is too wide to leave usable holes beside its pins, and is joined to them with male-to-female jumpers. Breadboard A carries the left foot and B the right foot. The 3V3 and GND pins of the ESP32 feed the rails of board A, and the rails of A are bridged to B. The signal junction of each divider goes to its own GPIO pin, and the full pin map is in Figure 2.

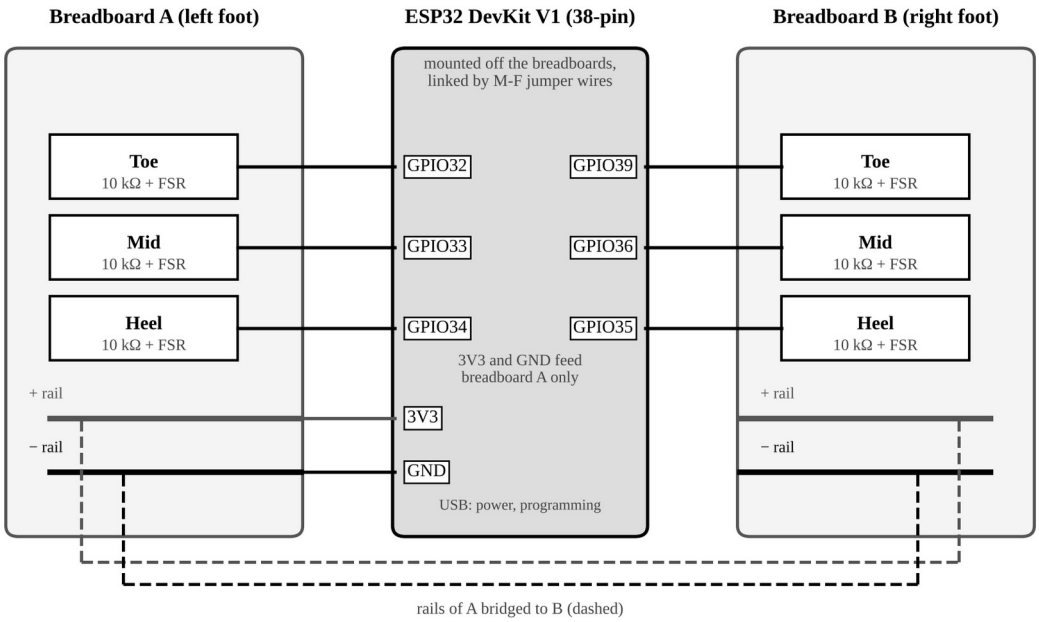



**Figure 3.** Hardware connection diagram of the two breadboards and the ESP32.

Figures 4 and 5 show the built prototype. The insoles are cut from cardboard, with the FSR pads taped at the toe, midfoot and heel, and are wired to the electronics box, which holds the ESP32, the two breadboards and the power bank.

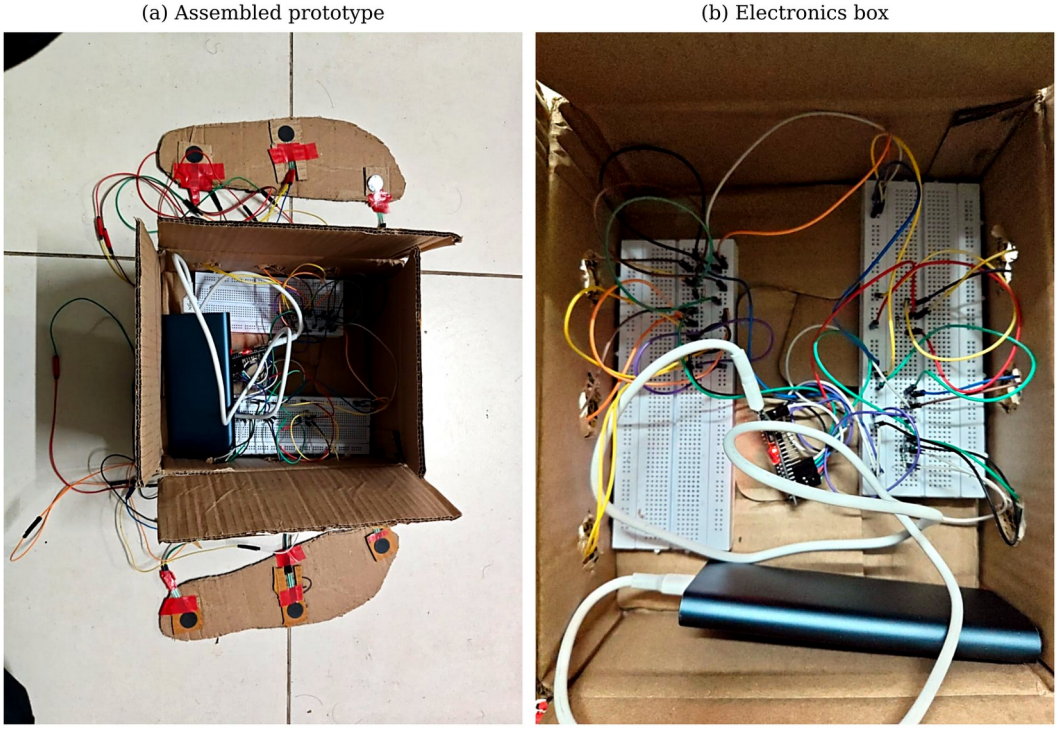



**Figure 4.** The built prototype: (a) overview and (b) the electronics box.

(a) Insole board 1 (b) Insole board 2

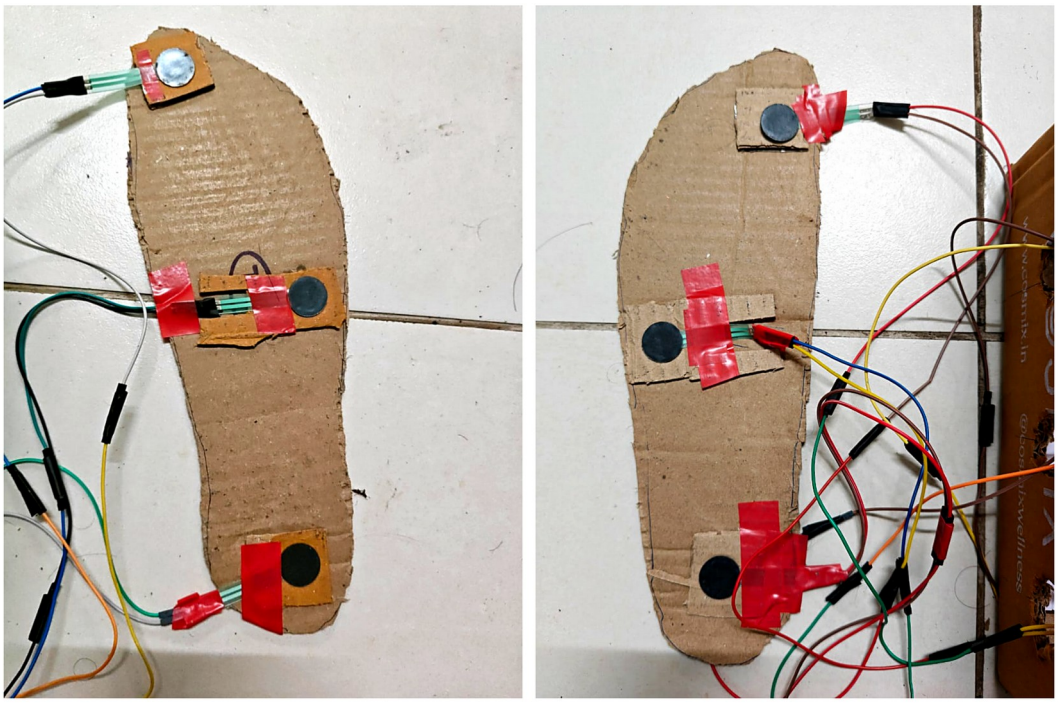


**Figure 5.** The two insole boards with three FSRs each.

### *5.3 Design decisions and observations*

- **ADC1 pins only.** All six pins (GPIO 32, 33, 34, 39, 36 and 35) are on ADC1, so WiFi can be switched on later without disturbing the ADC. GPIO 34, 35, 36 and 39 are input-only, which is fine for sensing.
- **3V3 only.** The dividers are powered from the 3V3 pin, never from VIN or 5 V, so the ADC input never exceeds its limit.
- **Pin map checked by pressing.** The map {32, 33, 34, 39, 36, 35} for left toe, left mid, left heel, right toe, right mid and right heel was verified by pressing each pad and watching which channel moved.
- **Unit-to-unit variation.** FSRs differ in sensitivity: swapping two sensors between channels showed that the behaviour follows the sensor and not the channel. Per-recording normalisation removes most of this for the model. For the foot map, a small per-pad gain equalises the two midfoot pads (factors 1.61 and 0.62).
- **Faulty and saturating pads.** A pad that reads near 100 counts when unloaded is shorted or faulty. A pad that bottoms out near 0 under body weight gives a flat-topped signal. Each pad was tested in a known-good channel before it was wired in, and the software repairs a clipped or dead pad (Section 7).

Figure 6 shows 12 s of raw output from the finished insole during stepping in place. The heel pads swing between about 4095 (foot lifted) and a few hundred counts (foot loaded), and the left and right feet alternate.

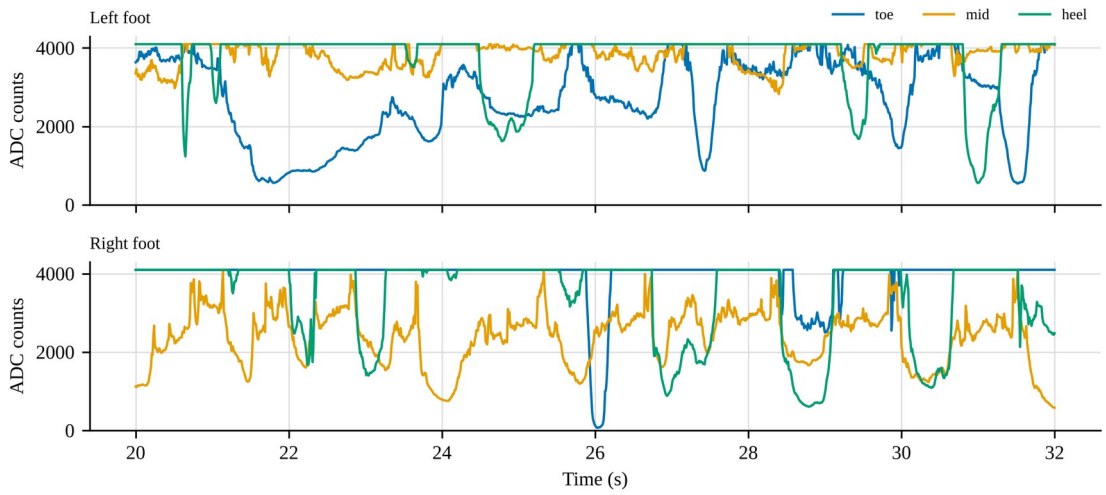



**Figure 6.** Raw ADC output of the six channels for 12 s of stepping in place (volunteer recording).

## 6. Software and Machine Learning Design

### *6.1 Tools and libraries*

**Table 3.** Software tools and libraries.

| Tool | Use |
|---|---|
| Arduino IDE 2.3.6, ESP32 core | Firmware: ADC reading, FreeRTOS sampler task, BluetoothSerial |
| Python 3.13, pyserial | Recorder (record.py): reads the serial or Bluetooth port and saves CSV files |
| NumPy | Preprocessing, gait metrics and the CNN forward pass in the cloud backend |
| FastAPI, Uvicorn, SQLite | Backend, HTTP API and database |
| HTML, CSS, JavaScript | Dashboard: caseload, patient view, foot map, playback and comparison |
| PyTorch, scikit-learn, pandas | Training and evaluation of the five models (GroupKFold, accuracy, F1, AUC) |

TensorFlow Lite, Edge Impulse and other TinyML toolchains were not used, because inference runs in NumPy on the backend. Section 6.6 explains how they would be used for on-device inference.

### *6.2 Data collection and dataset*

**Training data.** The PhysioNet gaitpdb dataset [11], [15] holds vertical ground-reaction-force recordings from 8 force sensors under each foot at 100 Hz, from people with Parkinson's disease and healthy controls. It merges three studies, named Ga, Ju and Si [12]-[14]. After parsing, 306 recordings from 165 subjects were used (93 with Parkinson's, 72 controls). Heel, midfoot and toe correspond to sensors 1, 4 and 8, which were inferred from when each sensor loads during stance and not checked against the sensor layout.

**Own recordings.** The insole was recorded with the PC recorder from team volunteers stepping in place for 60 s (and 20 s in one case), with no labels. These recordings were used to test the hardware, the metrics and the dashboard, and were never used to train a model.

### *6.3 Preprocessing*

The same windowing is applied to the training data and, with one extra step, to the insole data. Long recordings lose 2 s at each end, each channel is then z-scored over the recording, the signal is reduced from 100 Hz to 50 Hz by averaging pairs of samples, and cut into 2 s windows of 100 samples with 50% overlap. This gave 28,069 windows from gaitpdb, about 69% of them from people with Parkinson's, so a classifier that always answers Parkinson's would score about 0.69 accuracy. For the insole, the ADC reading is first converted to a load signal, because the training data are force, which rises with load, while the ADC value falls with load (Figure 7).

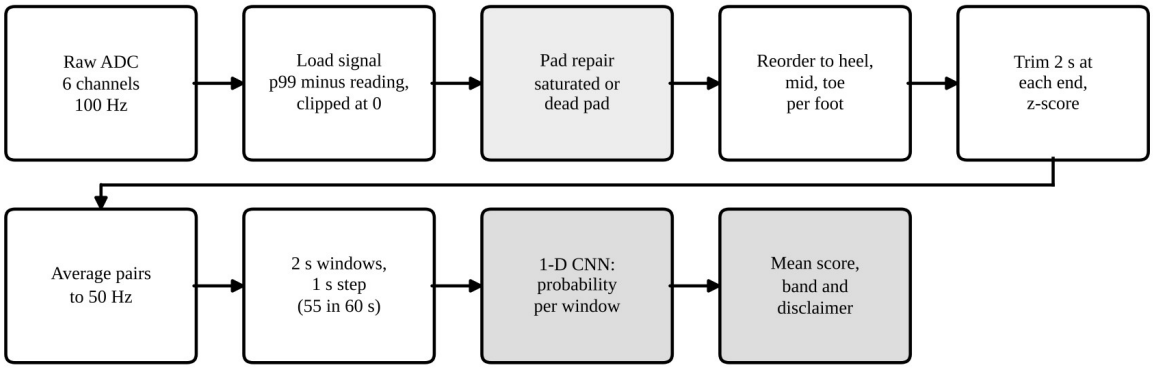



**Figure 7.** Scoring pipeline applied to an uploaded insole recording.

The load of a pad is the 99th percentile of its reading minus the reading, clipped at zero. The channels are reordered to the training order (heel, mid, toe per foot). A pad that bottoms out (more than 5% of its samples at or below 150 counts) is replaced by the mean of the other pads of the same foot, and a single dead pad per foot is filled the same way. If two pads of one foot are dead the recording is not scored. A 60 s recording gives 55 windows: 2,800 samples remain after trimming 2 s at each end and downsampling, and (2,800 − 100) / 50 + 1 = 55.

Three sensor sets were compared: S2 (heel and toe, 2 sensors per foot), S3 (heel, midfoot and toe, matching this insole) and S8 (all 8 sensors per foot).

### *6.4 Model selection and architecture*

Five models were trained on the same windows with the same settings. The CNN has three one-dimensional convolution blocks. The LSTM [18] has two layers of 64 units. The CNN-LSTM has two convolution blocks followed by an LSTM of 64 units. The Transformer [16] has a two-layer encoder with four heads and a feed-forward width of 128. The simplified Temporal Fusion Transformer (TFT) [17] is built from gated residual networks, per-channel embedding with variable selection, an LSTM encoder, gated skip connections and multi-head attention, and leaves out static covariates, known-future inputs and forecasting heads because this task does not use them. The TFT was run on S3 only.

The CNN was chosen for deployment because it had the best or joint-best macro F1 with the fewest parameters and the lowest latency (Section 9). Its architecture is in Figure 8 and Table 4.

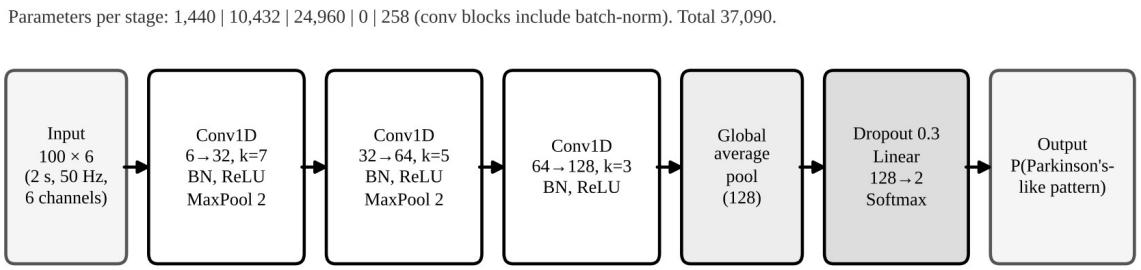



**Figure 8.** Architecture of the deployed 1-D CNN (S3, 6 input channels).

**Table 4.** Layers of the deployed CNN.

| Layer | Setting | Output shape | Parameters |
|---|---|---|---|
| Conv1D + BN + ReLU | 6 to 32, kernel 7 | 32 × 100 | 1,440 |
| MaxPool | size 2 | 32 × 50 | 0 |
| Conv1D + BN + ReLU | 32 to 64, kernel 5 | 64 × 50 | 10,432 |
| MaxPool | size 2 | 64 × 25 | 0 |
| Conv1D + BN + ReLU | 64 to 128, kernel 3 | 128 × 25 | 24,960 |
| Global average pool | over time | 128 | 0 |
| Dropout, Linear, Softmax | dropout 0.3, 128 to 2 | 2 | 258 |
| Total | | | 37,090 |

### *6.5 Training process*

**Table 5.** Training settings.

| Item | Setting |
|---|---|
| Input | 2 s windows, 100 samples at 50 Hz, z-scored per channel, 50% overlap |
| Dataset split | 5-fold GroupKFold by subject, so no person is in both training and test data |
| Loss | Cross-entropy, weighted by inverse class frequency |
| Optimiser | AdamW, weight decay 1e-4, cosine learning-rate schedule |
| Learning rate | CNN 1e-3, LSTM 2e-3, CNN-LSTM 1e-3, Transformer 5e-4, TFT 1e-3 |
| Regularisation | Dropout 0.3 before the classifier, gradient clipping at norm 1.0 |
| Batch size and epochs | 64 and 15 |
| Random seed | 0 (a single seed) |
| Robustness test | Leave-one-study-out on S3: train on two studies, test on the third |

The CNN deployed in the dashboard is the S3 model trained once on all subjects, with its weights exported as cnn_s3.npz.

### *6.6 Model optimisation for edge deployment*

**No optimisation was applied in this work.** The model was not quantised or pruned, and no TensorFlow Lite for Microcontrollers or Edge Impulse conversion was done. To make the cloud application light, the CNN was re-implemented in plain NumPy (convolution, batch norm, pooling and

a linear layer) so that the backend needs no PyTorch. It is written to match the PyTorch model in evaluation mode.

What the numbers say about moving it onto the ESP32 follows from the architecture and is calculated, not measured. The 37,090 weights take about 148 KB in 32-bit floating point (the exported file is 155 KB with batch-norm statistics) and about 37 KB if quantised to 8 bits, against 520 KB of SRAM on the ESP32 chip. The largest activation is the first convolution output (32 × 100 values, 12.8 KB in float32). One window needs about 1.26 million multiply-accumulates (conv 1: 134,400, conv 2: 512,000, conv 3: 614,400, plus the classifier). On-device inference is therefore plausible, but its latency, RAM use and power have not been measured here. The steps would be to export the model to int8 with post-training quantisation, convert it to a TensorFlow Lite for Microcontrollers array, and add a 2 s ring buffer for the six channels.

## 7. Implementation Details

### *7.1 Step-by-step implementation*

1. **Sensor bring-up.** Each FSR was tested in a known-good channel with a raw sketch that prints the six ADC values, and a pad with an idle reading near 100 was replaced.
2. **Firmware.** A FreeRTOS task pinned to core 1 reads the six channels every 10 ms and places each sample in a queue of 500. The main loop packs queued rows into text lines and sends them over Bluetooth (or USB serial if no client is connected).
3. **Recorder.** record.py opens the port, reads the rows for a set time, checks for gaps in the sample counter and saves a CSV file named by subject and condition.
4. **Backend.** The upload endpoint parses the CSV, calls the gait-metric and scoring functions and stores one row per session in SQLite.
5. **Dashboard.** The web page lists patients and sessions, shows metrics, the foot-pressure map with playback, a two-session comparison, alerts and clinical notes, and a card with the screening indicator.
6. **Model.** The CNN was trained offline, exported to cnn_s3.npz and loaded by the backend at start-up.

### *7.2 Firmware*

The sampler runs at a fixed period with vTaskDelayUntil, so the 100 Hz rate does not depend on how long Bluetooth transmission takes. The essential code is shown below.

```
const int PINS[6] = {32, 33, 34, 39, 36, 35};  //
L_toe,L_mid,L_heel,R_toe,R_mid,R_heel
const uint32_t PERIOD_MS = 10;                  // 100 Hz
struct Sample { uint32_t n; uint32_t t; uint16_t v[6]; };
QueueHandle_t q;

void samplerTask(void*) {
  TickType_t last = xTaskGetTickCount(); uint32_t idx = 0;
  for (;;) {
    vTaskDelayUntil(&last, pdMS_TO_TICKS(PERIOD_MS));
    Sample s; s.n = idx++; s.t = millis();
    for (int i = 0; i < 6; i++) s.v[i] = analogRead(PINS[i]);
```

```
    xQueueSend(q, &s, 0);
  }
}
void setup() {
  SerialBT.begin("insole"); analogReadResolution(12);
  q = xQueueCreate(500, sizeof(Sample));
  xTaskCreatePinnedToCore(samplerTask, "sampler", 4096, NULL, 3, NULL, 1);
}
```

### *7.3 Preprocessing and CNN inference (cloud)*

The backend turns the load signal into windows and runs the CNN with NumPy. The convolution is a sliding-window view followed by an einsum, which keeps the code short and fast enough for 55 windows.

```
def windows_from_signal(x, win_raw=200, ds=2, trim=200):
    T = win_raw // ds; step = T // 2
    if len(x) > 2*trim + 4*win_raw: x = x[trim:-trim]
    x = (x - x.mean(0)) / (x.std(0) + 1e-6)          # z-score per channel
    n = len(x) // ds; x = x[:n*ds].reshape(n, ds, -1).mean(1)   # 100 -> 50 Hz
    w = sliding_window_view(x, T, axis=0)[::step]     # 2 s windows, 50% overlap
    return w.transpose(0, 2, 1)                       # (N, T, C)

def _conv1d(x, w, b, pad):                            # x: (N, C, T)
    xp = np.pad(x, ((0,0),(0,0),(pad,pad)))
    win = sliding_window_view(xp, w.shape[2], axis=2)
    return np.einsum('nctk,ock->not', win, w, optimize=True) + b[None,:,None]

def band(score):                                      # shown on the dashboard
    return 'low' if score < 0.35 else 'high' if score > 0.5 else 'uncertain'
```

The recording's score is the mean probability over its windows. A score below 0.35 is shown as a low Parkinson-like pattern, between 0.35 and 0.5 as inconclusive, and above 0.5 as a Parkinson-like pattern shown, always with the line "This is only an indicator, not a diagnosis." The score depends on the shape of the signal and not on its gain, so sensor calibration and the midfoot gain cannot change it.

### *7.4 Gait metrics*

For stepping in place, each foot's load is scaled to its own range and the left-minus-right difference is formed, because the feet take turns carrying the weight. A step is counted each time this difference changes sign past a hysteresis band, which works even when the pads of the lifted foot never fully unload. Cadence, stride time, step and stride variability (coefficient of variation) and stance asymmetry follow from the step times. Pads that never press, or are pressed all the time, are dropped before this step.

### *7.5 Integration of the firmware with the model*

The microcontroller does not run the model. Integration is through the data format: the firmware's CSV row (n, t_ms, six counts) is the input of the cloud pipeline, and the order of channels in the firmware is mapped to the training order in one place (ORDER = [2, 1, 0, 5, 4, 3]). A change of pin map in the firmware therefore needs a change in that one list and nowhere else.

### *7.6 Communication protocols*

**Table 6.** Communication links.

| Link | Protocol | Detail |
|---|---|---|
| ESP32 to PC | Bluetooth Classic SPP | Device name "insole", appears as a COM port, text rows of about 40 bytes at 100 Hz |
| ESP32 to PC (fallback) | USB serial, 115,200 baud | Used for programming and when no Bluetooth client is connected |
| Recorder or browser to backend | HTTP, JSON and CSV upload | FastAPI endpoints for sessions, patients, notes, alerts and import |
| Backend to storage | SQLite | One row per session, with metrics, score and a short trace |

I2C and SPI are not used, since there is no inertial sensor. The sensors are analogue and are read directly by the ADC.

### *7.7 Dashboard views*

Figures 9 to 11 show the dashboard running on volunteer recordings, shown here as Volunteer A and B. The caseload page (Figure 9) lists each person with status, trend, last session, variability, a small trend line, asymmetry and the AI score. The patient page (Figure 10) shows the foot-pressure map, the step count and rate, step-time variability and contact-time asymmetry, the pressure trace with detected steps and the AI card with its disclaimer. The lower part of the page (Figure 11) holds the rehabilitation trend, the session list, the two-session comparison with its measures and the clinical notes. The numbers in these screenshots only show the interface working on our own recordings: they are not validated results.

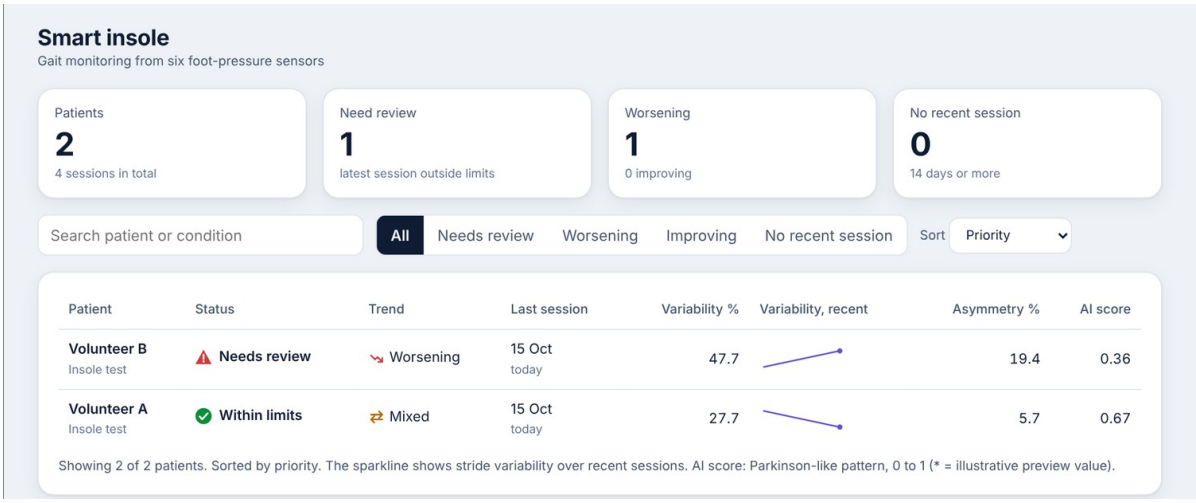



**Figure 9.** Dashboard caseload page.

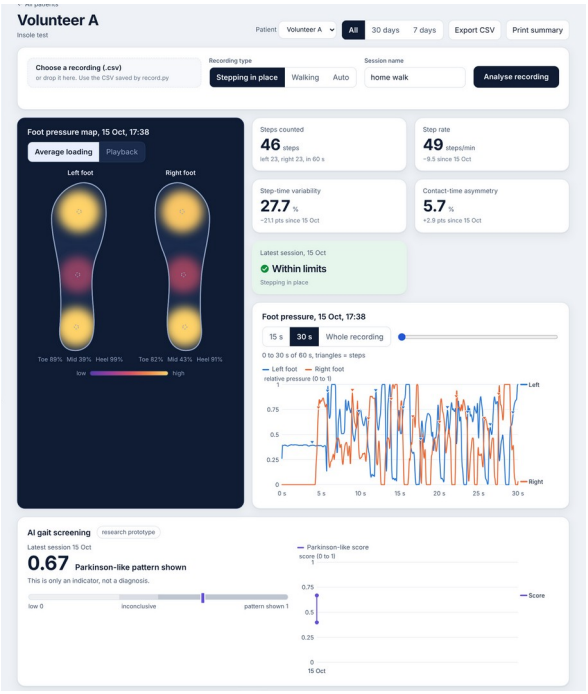



**Figure 10.** Patient page: foot-pressure map, gait metrics, pressure trace and AI card.

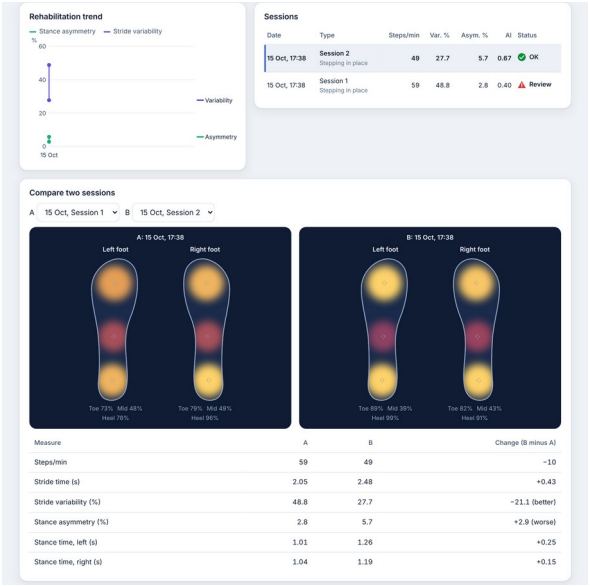



**Figure 11.** Session list, two-session comparison and clinical notes.

## 8. Experimental Setup

### *8.1 Test environment*

- **Model comparison.** Training and evaluation on gaitpdb with the settings of Table 5. The inference time per window of each model was measured on a notebook CPU, not on the ESP32.
- **Hardware sampling.** The insole breadboard prototype, powered over USB, recorded by the PC recorder over Bluetooth. Six recordings of 60 s and one of 20 s were taken from team volunteers stepping in place.
- **Backend timing.** The scoring and metric functions were timed in the cloud workspace used for development (2 virtual CPU cores, NumPy, no GPU) on the volunteer recordings.

### *8.2 How evaluation data were collected*

The classification results come from gaitpdb only, split by subject, so each test fold contains people the model never saw. The robustness test holds out one of the three studies at a time. The own-insole recordings have no ground truth: there were no patients, no manual step count and no reference system, so they test whether the hardware and pipeline work, not how accurate they are.

### *8.3 Performance metrics*

**Table 7.** Metrics and whether they were measured.

| Metric | How obtained | Status |
|---|---|---|
| Accuracy, macro F1, ROC AUC | 5-fold subject-wise cross-validation, leave-one-study-out | Measured (gaitpdb) |
| Subject accuracy | Window scores averaged per subject | Measured (gaitpdb) |
| Precision and recall per class | Macro F1 combines them | Reported as macro F1 only |
| Latency per window | CPU timing of each model; NumPy timing in the backend | Measured on a notebook CPU and a cloud CPU |
| Sampling rate and dropped samples | Timestamps and sample counter of the recordings | Measured on the insole |
| Model size and operations | Calculated from the architecture | Calculated (Section 6.6) |
| Memory on the ESP32 | Not measured | Not measured |
| Power on the ESP32 | Not measured | Not measured |
| Confusion matrix and ROC curve | Not produced | AUC values only (Section 9.4) |

### *8.4 Baselines*

Three baselines are used. The majority-class baseline (always answer Parkinson's) scores 0.69 accuracy on the pooled windows, and 0.82 on the Ju study alone. The five models are compared with each other under identical settings. For the hardware, the base paper [1] is the reference design: it uses two FSRs and an IMU per insole with an Arduino Uno and a mini-PC and reports 99.66% gait-event accuracy in healthy walking. It answers a different question (gait phase, not disease), so its result is not a baseline for ours. Table 8 compares the designs.

**Table 8.** Comparison of this work with the base paper design [1].

| Aspect | Yan et al. [1] | This work |
|---|---|---|
| Sensors per insole | 2 FSRs and a 9-axis IMU | 3 FSRs, no IMU |
| Controller | Arduino Uno R3 plus x86 mini-PC | ESP32 DevKit V1, cloud backend |
| Sampling rate | 300 Hz at the Arduino, model at 100 Hz | 100 Hz, no dropped samples |
| Learning model | MLP on IMU features with rule-based calibration | 1-D CNN on foot-pressure windows |
| Target | Five gait events in healthy walking | Parkinson's-like pattern versus control, plus gait metrics |
| Reported result | 99.66% accuracy, 32 ms time error (5 subjects) | Macro F1 0.759 on gaitpdb, not validated on the insole |
| Wireless and cloud | Not described | Bluetooth, cloud dashboard |

## 9. Results and Discussion

### *9.1 Classification results on gaitpdb*

Table 9 gives the mean over the five subject-wise folds, with accuracy as mean ± standard deviation across folds. Inference time is for one window on a CPU. Figure 12(a) shows macro F1.

**Table 9.** Subject-wise cross-validation results (mean over 5 folds).

| Sensors | Model | Accuracy | Macro F1 | AUC | Subject accuracy | Parameters | Inference (ms) |
|---|---|---|---|---|---|---|---|
| S2 | CNN | 0.799 ± 0.060 | 0.772 | 0.874 | 0.788 | 36,642 | 0.38 |
| S2 | LSTM | 0.761 ± 0.024 | 0.721 | 0.818 | 0.758 | 51,330 | 0.83 |
| S2 | CNN-LSTM | 0.786 ± 0.044 | 0.753 | 0.844 | 0.788 | 44,578 | 0.70 |
| S2 | Transformer | 0.739 ± 0.030 | 0.706 | 0.799 | 0.758 | 73,922 | 1.10 |
| S3 | CNN | 0.790 ± 0.051 | 0.759 | 0.866 | 0.794 | 37,090 | 0.55 |
| S3 | LSTM | 0.779 ± 0.037 | 0.750 | 0.838 | 0.770 | 51,842 | 0.85 |
| S3 | CNN-LSTM | 0.791 ± 0.057 | 0.759 | 0.862 | 0.776 | 44,898 | 0.69 |
| S3 | Transformer | 0.763 ± 0.031 | 0.729 | 0.823 | 0.782 | 74,050 | 1.16 |
| S3 | TFT (simplified) | 0.762 ± 0.066 | 0.730 | 0.816 | 0.770 | 53,466 | 3.01 |
| S8 | CNN | 0.775 ± 0.043 | 0.744 | 0.853 | 0.776 | 39,330 | 0.41 |
| S8 | LSTM | 0.757 ± 0.077 | 0.728 | 0.802 | 0.745 | 54,402 | 0.87 |
| S8 | CNN-LSTM | 0.784 ± 0.045 | 0.752 | 0.843 | 0.782 | 46,498 | 0.65 |
| S8 | Transformer | 0.744 ± 0.040 | 0.709 | 0.817 | 0.745 | 74,690 | 0.99 |

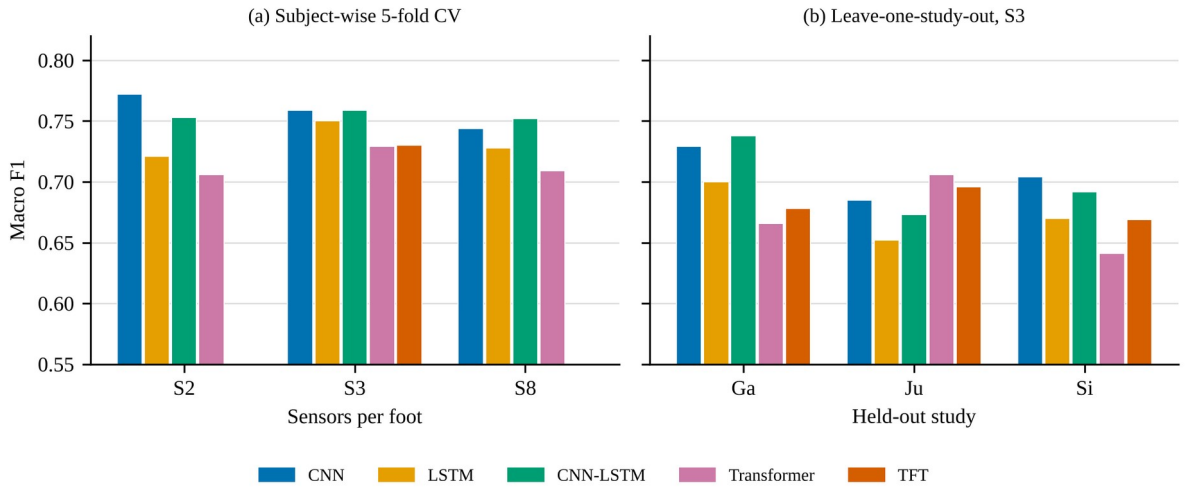



**Figure 12.** Macro F1 of each model. (a) Subject-wise cross-validation for each sensor set. (b) Leave-one-study-out test on S3.

The CNN has the highest macro F1 on S2 (0.772) and ties the CNN-LSTM on S3 (0.759); on S8 the CNN-LSTM is slightly ahead (0.752 against 0.744). The CNN has the fewest parameters (about 37,000) and the lowest inference time (0.4 to 0.55 ms per window). The Transformer scores lowest in every sensor set with twice the parameters, and the TFT matched the Transformer (0.730 against 0.729) while being the slowest at about 3 ms per window, so its extra components gave no benefit on this task. Going from 8 sensors to 3 per foot did not lower performance (CNN macro F1 0.744, 0.759 and 0.772 for S8, S3 and S2), which supports the three-sensor insole. The differences between models are small compared with the spread between folds (accuracy standard deviation 0.02 to 0.08), so the ranking is suggestive and not conclusive.

### *9.2 Robustness across studies*

Each model was trained on two studies and tested on the third (S3, same settings). Table 10 gives macro F1 for each held-out study and the mean over the three splits.

**Table 10.** Leave-one-study-out results on the S3 sensor set.

| Model | F1, hold out Ga | F1, hold out Ju | F1, hold out Si | Mean accuracy | Mean macro F1 | Mean AUC |
|---|---|---|---|---|---|---|
| CNN | 0.729 | 0.685 | 0.704 | 0.743 ± 0.034 | 0.706 | 0.805 |
| LSTM | 0.700 | 0.652 | 0.670 | 0.710 ± 0.035 | 0.674 | 0.779 |
| CNN-LSTM | 0.738 | 0.673 | 0.692 | 0.740 ± 0.041 | 0.701 | 0.791 |
| Transformer | 0.666 | 0.706 | 0.641 | 0.712 ± 0.079 | 0.671 | 0.764 |
| TFT (simplified) | 0.678 | 0.696 | 0.669 | 0.728 ± 0.063 | 0.681 | 0.753 |

Every model scored lower than in the subject-wise folds (CNN accuracy 0.743 against 0.790, macro F1 0.706 against 0.759; Figure 13a), which suggests that part of the earlier score reflects patterns specific to each study. The CNN and CNN-LSTM transferred best, and the CNN had the highest mean AUC (0.805). The Transformer and TFT had the widest spread across studies. The studies differ in class balance (67%, 82% and 55% Parkinson's windows in Ga, Ju and Si), so accuracy should not be compared across them; in Ju, always answering Parkinson's gives 0.82, above every model, which is why macro F1 and AUC are the main measures. With three splits and one random seed, the gaps

between models are small, and the finding that holds for all of them is the drop when the study changes. The convolutional models are not exactly repeatable on a GPU: a repeat run moved their mean macro F1 by less than 0.01, so the order of the CNN and CNN-LSTM is a tie.

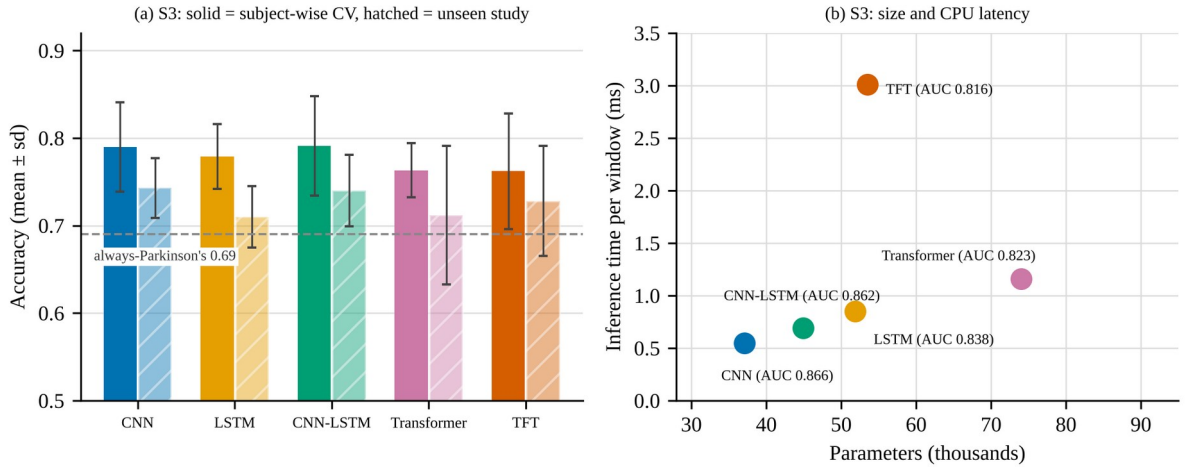



**Figure 13.** (a) Accuracy on S3 for subject-wise cross-validation and for an unseen study, against the always-Parkinson's baseline. (b) Parameters and CPU inference time per window.

### *9.3 Real-time performance*

Table 11 gives the timing and size figures. The insole sampled at exactly 100 Hz: in all seven recordings (about 6,000 rows per minute) the timestamps are 10 ms apart with no standard deviation and the sample counter has no gaps. The CNN needs about 0.55 ms per window on a notebook CPU, and the full scoring of a 60 s recording, including preprocessing and 55 windows, takes about 36 ms in the backend, so a score appears essentially at the moment the upload ends. Inference on the ESP32 itself was not measured.

**Table 11.** Real-time performance.

| Quantity | Value | Where measured |
|---|---|---|
| Sampling rate, dropped samples | 100 Hz, none in 7 recordings | Insole |
| CNN, one window | 0.55 ms | Notebook CPU |
| CNN forward pass, 55 windows | 15 ms (0.27 ms per window) | Cloud CPU, NumPy |
| Full scoring of a 60 s recording | 36 ms (median of 10 runs) | Cloud CPU, NumPy |
| Gait metrics of a 60 s recording | 15 to 26 ms | Cloud CPU, NumPy |
| Backend process memory | about 40 MB peak | Cloud, includes Python and NumPy |
| Model weights | 148 KB float32, about 37 KB int8 | Calculated |
| Operations per window | about 1.26 million MACs | Calculated |
| ESP32 inference time, RAM, power | Not measured | - |

For power, only a rough indication can be given: the ESP32 draws typically of the order of 100 mA with Bluetooth active, which a small power bank can supply for many hours, but no current was measured on the prototype.

### *9.4 Confusion matrix and ROC curve*

A confusion matrix and ROC curve were not produced. The supplied evaluation output holds per-fold accuracy, macro F1, AUC and subject accuracy, but not the stored window predictions needed to draw them, and the gaitpdb data could not be downloaded in the workspace used to prepare this document. The AUC values (Tables 9 and 10) summarise the ROC curve and the macro F1 averages the two per-class F1 scores. Re-running the evaluation with the predictions saved is a small change and is listed under future work.

### *9.5 Gait metrics from the insole*

Table 12 gives the dashboard's metrics for the volunteer recordings. They show that the pipeline works on real insole data; they are not accuracy results, because no reference step count exists. The variability is high because untrained volunteers marched in place, and volunteers who were not patients showing 20 to 30% asymmetry in some recordings suggests that the detector and the sensor differences still add noise.

**Table 12.** Metrics for seven volunteer recordings (stepping in place).

| ID | Active time (s) | Steps | Cadence (steps/min) | Stride time (s) | Stride CV (%) | Asymmetry (%) | Time (ms) |
|---|---|---|---|---|---|---|---|
| V1 | 56.2 | 46 | 49.1 | 2.48 | 27.7 | 5.7 | 26 |
| V2 | 56.7 | 43 | 45.5 | 2.63 | 54.2 | 29.2 | 24 |
| V3 | 54.2 | 45 | 49.8 | 2.42 | 26.6 | 24.2 | 16 |
| V4 | 55.6 | 60 | 64.8 | 1.87 | 47.7 | 19.4 | 15 |
| V5 | 56.0 | 57 | 61.1 | 1.95 | 22.5 | 1.1 | 16 |
| V6 | 57.4 | 56 | 58.6 | 2.05 | 48.8 | 2.8 | 15 |
| V7 | 16.8 | 13 | 46.5 | 2.71 | 32.5 | 12.9 | 11 |

### *9.6 Discussion*

**What worked.** The hardware reached its design goal: six channels at a steady 100 Hz over Bluetooth with no gaps. Per-recording normalisation and the pad repair made recordings from different sensor units usable by the same model. A small CNN was as good as the larger sequence models and much cheaper, and three sensors per foot lost nothing against eight. The whole path from the insole to a dashboard card runs in well under a second.

**Limitations.** The model was trained on a different insole (8 force sensors per foot, mostly older adults) and has not been validated on this one. On our own marching-in-place recordings the score was unstable: for one recording it moved between about 0.2 and 0.8 under small changes that should not matter, and clipped, stuck or dead pads raised the score. There are no labelled patients, so no accuracy can be claimed on the insole, and every score card carries the line "This is only an indicator, not a diagnosis." Other limits are the single random seed, the inferred sensor positions for S3, possible differences in age and walking speed between patients and controls in gaitpdb, and the fact that stroke gait was not evaluated.

**Error analysis.** Three sources of error stand out. (i) Domain shift: the macro F1 drops by 0.05 to 0.08 on an unseen study, and the insole is a further step away from the training sensors, with a resistive, non-linear response, different pad positions and stepping in place instead of walking. (ii) Class imbalance:

with 69% Parkinson's windows overall and 82% in Ju, accuracy is a weak measure and a model can look good by leaning towards the majority class. (iii) Sensor artefacts: a pad that bottoms out gives a flat-topped waveform that the model has never seen in healthy people and which pushes the score up, which is why the pipeline repairs such pads. Because of these, the dashboard shows a score in three bands with a one-line disclaimer and not a label.

## 10. Conclusion

We built a six-FSR smart insole on an ESP32 that records at a steady 100 Hz over Bluetooth, a cloud application that turns the recordings into gait metrics, foot-pressure maps and alerts, and a screening indicator based on a small 1-D CNN. On PhysioNet gaitpdb, the CNN matched or beat a CNN-LSTM, an LSTM, a Transformer and a simplified TFT with the fewest parameters (macro F1 0.759 and AUC 0.866 on three sensors per foot), and three sensors per foot were enough. All models lost 0.05 to 0.08 macro F1 on a study they had not seen, and the model is not validated on our insole, so the output is presented as an indicator and never as a diagnosis.

Future work follows from the limits above. (1) Collect labelled recordings from the insole, with ethics approval, and fine-tune or re-train on them. (2) Quantise the CNN to 8 bits, convert it for TensorFlow Lite for Microcontrollers, run it on the ESP32 and measure latency, RAM and current. (3) Save window predictions to produce confusion matrices and ROC curves, and repeat the experiments with several seeds. (4) Add an inertial sensor for step events, and evaluate stroke gait. (5) Replace the breadboards with a soldered or flexible circuit in a real insole, and add a login and encryption to the cloud application before any real patient data is used.

## References


[1] L. Yan, G. Wei, Z. Hu, H. Xiu, Y. Wei and L. Ren, "Low-cost multisensor integrated system for online walking gait detection," Journal of Sensors, vol. 2021, Article ID 6378514, 2021. doi:10.1155/2021/6378514.

[2] I. El Maachi, G.-A. Bilodeau and W. Bouachir, "Deep 1D-Convnet for accurate Parkinson disease detection and severity prediction from gait," Expert Systems with Applications, Article 113075, 2020. doi:10.1016/j.eswa.2019.113075.

[3] D. M. D. Nguyen, M. Miah, G.-A. Bilodeau and W. Bouachir, "Transformers for 1D signals in Parkinson's disease detection from gait," arXiv:2204.00423, 2022.

[4] M. Meral and F. Ozbilgin, "Parkinson's disease diagnosis and severity assessment from gait signals via Bayesian-optimized deep learning," Diagnostics, vol. 15, no. 16, Article 2046, 2025. doi:10.3390/diagnostics15162046.

[5] E.-S. Park, X. Liu, H.-J. Hwang and C.-H. Han, "Deep convolutional neural network-based detection of gait abnormalities in Parkinson's disease using fewer plantar sensors in a smart insole," Biosensors, vol. 16, no. 1, Article 40, 2026. doi:10.3390/bios16010040.

[6] D. Boucharas et al., "Smart insole: a gait analysis monitoring platform targeting Parkinson disease patients based on insoles," arXiv:2212.00109, 2022.

[7] D. Movrin, S. Chatpun, M. Simic and T. Sanghan, "Design and evaluation of a smart insole system for real-time gait and plantar pressure monitoring," Advanced Technologies and Materials, vol. 50, no. 2, pp. 31-37, 2025. doi:10.24867/ATM-2025-2-005.

[8] Z. A. Madhiha, A. Mazumder and S. M. Hiam, "A cost-effective, stand-alone, and real-time TinyML-based gait diagnosis unit aimed at lower-limb robotic prostheses and exoskeletons," arXiv:2411.08474, 2024.

[9] B. Latsch et al., "Sensor insoles: a review," arXiv:2509.00260, 2025.

[10] H. A. Abushahla, D. Varam, A. J. N. Panopio and M. I. AlHajri, "Neural network quantization for microcontrollers: a comprehensive survey of methods, platforms, and applications," arXiv:2508.15008, 2025.

[11] A. L. Goldberger et al., "PhysioBank, PhysioToolkit, and PhysioNet: components of a new research resource for complex physiologic signals," Circulation, vol. 101, no. 23, pp. e215-e220, 2000.

[12] G. Yogev, N. Giladi, C. Peretz, S. Springer, E. S. Simon and J. M. Hausdorff, "Dual tasking, gait rhythmicity, and Parkinson's disease: which aspects of gait are attention demanding?" European Journal of Neuroscience, vol. 22, no. 5, pp. 1248-1256, 2005.

[13] J. M. Hausdorff, J. Lowenthal, T. Herman, L. Gruendlinger, C. Peretz and N. Giladi, "Rhythmic auditory stimulation modulates gait variability in Parkinson's disease," European Journal of Neuroscience, vol. 26, no. 8, pp. 2369-2375, 2007.

[14] S. Frenkel-Toledo, N. Giladi, C. Peretz, T. Herman, L. Gruendlinger and J. M. Hausdorff, "Effect of gait speed on gait rhythmicity in Parkinson's disease: variability of stride time and swing time respond differently," Journal of NeuroEngineering and Rehabilitation, vol. 2, Article 23, 2005.

[15] PhysioNet, "Gait in Parkinson's Disease (gaitpdb), version 1.0.0." https://physionet.org/content/gaitpdb/1.0.0/

[16] A. Vaswani et al., "Attention is all you need," Advances in Neural Information Processing Systems, vol. 30, 2017.

[17] B. Lim, S. O. Arik, N. Loeff and T. Pfister, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," International Journal of Forecasting, vol. 37, no. 4, pp. 1748-1764, 2021.

[18] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.